

Módulo 12 – Capítulo 08 – Sección 01

Stack de observabilidad del proyecto: trazas, métricas, logs y alertas

La observabilidad de un sistema de IA en producción no es una capa que se añade después del despliegue — es la infraestructura que hace posible operar con confianza en lugar de con esperanza. Un sistema que no se puede observar no se puede mejorar, no se puede diagnosticar cuando falla y no se puede demostrar que funciona correctamente ante stakeholders que necesitan evidencia. La observabilidad convierte el sistema de una caja negra que "parece funcionar" en un sistema transparente cuyo comportamiento puede describirse con precisión en cualquier momento.

El stack de observabilidad del proyecto implementa los tres pilares de la observabilidad moderna más una capa de alertas, siguiendo el estándar OpenTelemetry para garantizar que la instrumentación es vendor-neutral y puede exportarse a cualquier backend sin cambiar el código de la aplicación. Esta independencia del vendor no es una consideración abstracta: si el equipo decide migrar de Grafana Tempo a Jaeger para las trazas, o de Grafana Loki a Elasticsearch para los logs, el código de instrumentación OpenTelemetry no cambia — solo cambia la configuración del OTEL Collector.

Las trazas distribuidas capturan el ciclo de vida completo de cada petición al sistema, desde que el cliente HTTP envía la request hasta que recibe la respuesta. El SDK de OpenTelemetry para Python instrumenta automáticamente FastAPI (spans HTTP por endpoint), httpx (spans para cada llamada a API externa — OpenAI, Cohere), SQLAlchemy (spans para cada query a PostgreSQL) y el cliente de Qdrant (spans para cada búsqueda en la base vectorial). Además de la instrumentación automática, el código del agente añade spans custom para cada paso del ciclo ReAct — `agent.thought_{n}`, `agent.tool_call.{tool_name}`, `agent.final_answer` — que no son instrumentables automáticamente porque son específicos de la lógica del agente. Las trazas se exportan a Grafana Tempo a través del OTEL Collector.

Las métricas se exponen via Prometheus scraping del endpoint `/metrics` de FastAPI mediante el middleware `prometheus_fastapi_instrumentator`. Las métricas automáticas incluyen histogramas de latencia HTTP por endpoint y método, contadores de requests por status code, y gauge de requests en proceso. Las métricas custom incluyen: `llm_tokens_total` (contador de tokens consumidos por modelo y por tipo — prompt/completion), `ragas_faithfulness` (gauge de la métrica de faithfulness calculada sobre el sample de evaluación online), `agent_iterations_total` (histograma del número de iteraciones por ejecución del agente) y `tool_call_duration_seconds` (histograma de duración por herramienta). Estas métricas custom son las que el Capítulo 7 usa para el framework de evaluación continua.

Los logs estructurados se emiten en formato JSON mediante el módulo `logging` de Python configurado con `python-json-logger`. Cada log incluye campos estándar (`level`, `timestamp`, `logger_name`) más campos específicos del dominio (`trace_id`, `span_id`, `user_id`, `tool_name`, `tokens_used`, `error_code`). La presencia de `trace_id` y `span_id` en los logs es crítica: permite que Grafana Loki correlacione un log específico con la traza que lo generó, haciendo posible el flujo de debugging "alerta → traza → log" sin saltar manualmente entre herramientas. Los logs se exportan a Grafana Loki a través del OTEL Collector con el exporter de logs de OpenTelemetry.

Componentes del stack de observabilidad

- **OpenTelemetry SDK:** instrumentación automática de FastAPI, httpx, SQLAlchemy y cliente Qdrant; instrumentación custom del ciclo ReAct del agente con spans `agent.thought`, `agent.tool_call`, `agent.final_answer`; propagación automática de trace context entre servicios via headers W3C TraceContext.
- **OTEL Collector:** recibe trazas, métricas y logs; aplica transformaciones (filtrado, sampling, enriquecimiento con metadatos del entorno); exporta a Grafana Tempo (trazas), Prometheus (métricas) y Grafana Loki (logs).
- **Grafana Tempo:** almacenamiento y visualización de trazas distribuidas; correlación con métricas de Prometheus (click en un span → ver métrica correspondiente) y con logs de Loki (click en un span → ver logs de esa ejecución).
- **Prometheus + Grafana:** colección de métricas de FastAPI (automáticas) y del dominio (custom); dashboards con correlación entre métricas de calidad RAG, rendimiento del agente y métricas de infraestructura.
- **Grafana Loki:** almacenamiento de logs estructurados JSON; LogQL para queries complejas ("todos los logs de error del agente en las últimas 2 horas donde

tool_name=search_knowledge_base"); correlación con trazas via trace_id.

Nota del Arquitecto: El parámetro de sampling de trazas merece atención especial. En producción con 5000 queries/día, almacenar el 100% de las trazas en Grafana Tempo genera aproximadamente 15GB de datos diarios — caro y difícil de analizar. La estrategia de sampling correcta es: 100% de las trazas de requests que retornan error (para debugging de incidentes), 10% de las trazas de requests normales (muestra estadísticamente significativa para análisis de rendimiento), y 100% de las trazas de requests marcados como *suspicious* por el clasificador de intent (para análisis de seguridad). Este sampling adaptativo reduce el volumen de almacenamiento en un 85% sin perder la cobertura de debugging necesaria.

La instrumentación con OpenTelemetry debe comenzar desde el primer commit del proyecto, no añadirse después. Instrumentar retroactivamente un sistema existente es una tarea que puede tomar días y que frecuentemente produce cobertura incompleta — los spans custom del agente requieren conocer la estructura interna del grafo LangGraph, que es más difícil de entender después de que el código está escrito. El Capítulo 8 continúa con el tracing distribuido en detalle — la anatomía de una traza completa desde el cliente hasta el modelo y de vuelta.

Para recordar: Un stack de observabilidad solo es útil si los tres pilares están correlacionados — la capacidad de ir de una alerta de latencia alta a la traza específica que causó el problema, y de ahí a los logs de esa ejecución, es lo que permite diagnosticar incidentes en minutos en lugar de horas.

Módulo 12 – Capítulo 08 – Sección 02

Tracing distribuido: seguimiento de una petición desde el usuario hasta el modelo y de vuelta

Una traza distribuida es la historia completa de una petición al sistema — desde el primer byte que llega al endpoint de FastAPI hasta el último byte de la respuesta que sale hacia el cliente. Sin una traza, debuggear una petición que tardó 4.5 segundos (cuando el SLA es 3s) requiere revisar los logs de FastAPI, los logs del agente, los logs del cliente de Qdrant, los logs del cliente de Cohere y los logs del cliente de OpenAI, buscando manualmente las entradas correspondientes a esa petición específica por timestamp. Con una traza, el ingeniero de guardia abre Grafana Tempo, busca la traza por `trace_id` (disponible en el header de respuesta `X-Trace-ID`), y ve en un gráfico de Gantt exactamente cuánto tiempo pasó en cada etapa: "el reranker de Cohere tardó 1.8 segundos (4x más que la mediana), que es el cuello de botella de esta petición".

La anatomía de una traza completa del sistema integrador tiene una estructura jerárquica de spans. El **span raíz** `http.request` es creado automáticamente por la instrumentación OpenTelemetry de FastAPI cuando llega la petición al endpoint `/query`. Incluye atributos: `http.method=POST`, `http.url=/query`, `http.status_code=200`, `http.user_agent`, `user_id` (extraído del JWT por el middleware de autenticación) y `session_id`. Su duración es la latencia end-to-end de la petición.

Dentro del span raíz, los **spans hijo de nivel 1** son: `auth.validate_jwt` (verificación de firma RS256 del token, extracción de claims), `input.validate` (schema Pydantic + lista negra + clasificador de intent), `agent.run` (span contenedor del ciclo ReAct completo) y `output.filter` (PII scanning de la respuesta). `auth.validate_jwt` e `input.validate` son rápidos (< 20ms cada uno) y sus atributos de error son los que diagnostican problemas de autenticación y validación. `agent.run` es el span más largo y tiene la estructura más compleja.

Dentro de `agent.run`, los **spans del ciclo ReAct** se numeran por iteración. El span `agent.thought_1` captura el primer razonamiento del LLM con atributos `model=gpt-4o-`

2024-08-06, `prompt_tokens=350`, `thought_text=<hash>` (el texto del Thought se hashea por privacidad en producción, disponible en texto plano solo en modo debug). El span `agent.tool_call.search_knowledge_base_1` captura la primera llamada a la herramienta de búsqueda, con sus propios spans hijo: `embedding.query` (llamada a text-embedding-3-small), `qdrant.search` (búsqueda híbrida RRF) y `cohere.rerank` (reranking de los 20 candidatos). Si el agente necesita más de una iteración, aparecen `agent.thought_2`, `agent.tool_call.search_knowledge_base_2`, etc. El span `agent.final_answer` captura la generación de la respuesta final con `model=gpt-4o-2024-08-06`, `prompt_tokens=2400`, `completion_tokens=250`, `finish_reason=stop`.

El flow de debugging con Grafana Tempo para un incidente de alta latencia es: recibir la alerta de Grafana Alerting ("p95_latency > 3500ms en los últimos 10 minutos"), abrir el panel de latencia en el dashboard operativo, hacer click en el pico de latencia para ver las trazas correspondientes a ese período, seleccionar las trazas con duración > 3500ms, examinar el span tree de las trazas más lentas para identificar qué span hijo es el cuello de botella, hacer click en el span problemático para ver los logs de Loki asociados a esa ejecución específica. Este flow completo toma entre 5 y 10 minutos con el stack de observabilidad correctamente configurado.

Spans del trace distribuido

- `http.request` (raíz): método, URL, status_code, user_id, session_id, ip_hash — duración total de la petición.
- `auth.validate_jwt`: verificación RS256, extracción de claims, validación de expiración — atributos `token_valid`, `roles`, `error_code` si falla.
- `input.validate`: schema Pydantic, lista negra, clasificador de intent — atributos `classification=safe/suspicious/malicious`, `validation_error` si aplica.
- `agent.run`: span contenedor del ciclo ReAct — atributos `iterations_count`, `task_completed`, `total_tokens`, `estimated_cost_usd`.
- `agent.thought_{n}`: razonamiento del LLM en cada iteración — atributos `model`, `prompt_tokens`, `thought_hash` (hash por privacidad, plain text en modo debug).
- `tool.search_knowledge_base_{n}`: ejecución de herramienta con sub-spans `embedding.query`, `qdrant.search`, `cohere.rerank` — atributos `query_length`, `results_count`, `max_relevance_score`.
- `llm.generate`: generación de respuesta final — atributos `model`, `prompt_tokens`, `completion_tokens`, `finish_reason`, `estimated_cost_usd`.

- `output.filter` : PII scanning — atributos `pii_detected` (boolean), `patterns_matched` (count, sin detalle del contenido detectado).

Nota del Arquitecto: El atributo `estimated_cost_usd` en los spans del agente y del LLM es una adición custom que el stack estándar de OpenTelemetry no incluye. Se calcula en el código de la aplicación multiplicando los tokens consumidos por las tarifas vigentes, ambas disponibles al momento de generar el span. Este atributo hace posible el análisis de costo por traza en Grafana Tempo: el ingeniero puede filtrar las trazas más costosas del día y analizar qué tipos de preguntas generan el mayor costo, sin necesidad de correlacionar manualmente las trazas con los datos de facturación de OpenAI.

El tracing distribuido requiere que el contexto de traza (`trace_id` y `span_id`) se propague correctamente entre todos los componentes del sistema. El SDK de OpenTelemetry propaga automáticamente el contexto en headers HTTP via el estándar W3C TraceContext (`traceparent: 00-{trace_id}-{span_id}-01`) en las llamadas salientes de httpx — esto garantiza que cuando FastAPI llama a la API de OpenAI o a Cohere, esas llamadas aparecen como spans hijo en la misma traza, no como trazas separadas sin relación.

Para recordar: El tracing distribuido no es solo para debugging — es la fuente de verdad sobre la latencia de cada componente en producción, y es el único mecanismo para identificar qué etapa del pipeline está causando una degradación de P95 sin instrumentación adicional.

Módulo 12 – Capítulo 08 – Sección 03

Dashboard operativo: visualización de métricas clave en tiempo real

Un dashboard de observabilidad efectivo no es una colección de paneles con todas las métricas disponibles — es una narrativa visual del estado del sistema que permite al equipo de operaciones diagnosticar en segundos si el sistema funciona correctamente y, si no, en qué dimensión está fallando. El diseño del dashboard refleja las prioridades del equipo: qué información es la primera que necesita un ingeniero de guardia que recibe una alerta a las 2am, y qué información necesita el equipo de producto que revisa el estado del sistema en la reunión semanal.

El dashboard operativo del sistema integrador en Grafana organiza las métricas en cinco filas que cuentan la historia del sistema de lo más urgente a lo más estratégico. La primera fila es el **estado actual** con cuatro stat panels semafóricos: peticiones/minuto (verde < 200, amarillo 200-400, rojo > 400 — indica sobrecarga), latencia P95 (verde < 2.5s, amarillo 2.5-3s, rojo > 3s), error rate (verde < 0.5%, amarillo 0.5-1%, rojo > 1%) y costo estimado por hora en USD (verde dentro del presupuesto, rojo si supera 1.5x el promedio). Esta fila es la primera que mira el ingeniero de guardia — si todos están en verde, el sistema está operando correctamente; si alguno está en rojo, hay un incidente activo.

La segunda fila es **calidad del sistema** con dos gráficos de serie temporal de 24 horas: faithfulness y answer_relevance medidos sobre el sample de evaluación online del 5% del tráfico. Cada gráfico tiene una línea de umbral roja en 0.80 (faithfulness) y 0.75 (answer_relevance) — los umbrales de alerta, inferiores a los de producción para dar margen estadístico en la evaluación online. Una caída por debajo de estas líneas dispara una alerta de WARNING que notifica al equipo en Slack. Esta fila es la que el equipo de producto revisa semanalmente para evaluar si la calidad del sistema está mejorando o degradándose.

La tercera fila es **rendimiento del pipeline** con un heatmap de latencia por hora del día (permite identificar patrones horarios de degradación — si el P95 aumenta sistemáticamente a las 14h, puede ser un pico de uso del equipo de producto), un histograma de latencia end-to-

end actual, y un gráfico de serie temporal de throughput en req/s con línea de capacidad máxima verificada.

La cuarta fila es **comportamiento agéntico** con tres paneles: un histograma de `iterations_per_task` en los últimos 60 minutos (distribución esperada: moda en 2, larga cola a la izquierda en 1 y a la derecha en 5 indica posible degradación del razonamiento), un bar chart de `tool_call_count` por herramienta (detecta cambios en los patrones de uso de herramientas), y la `task_completion_rate` en ventana deslizante de 1 hora (umbral de alerta: < 70%).

La quinta fila es **seguridad y costos** con contadores de rechazos por tipo (`injection_detected`, `rate_limit`, `auth_failure`, `schema_validation`), el costo acumulado del día en USD con proyección de fin de mes, y un panel de `allowed_document_types_violations` (debería ser siempre 0 en operación normal — cualquier valor distinto de cero requiere investigación inmediata).

Paneles del dashboard operativo

- **Fila 1 — Estado actual:** stat panels con semáforos para peticiones/min, latencia P95, error rate y costo USD/hora; diseñados para diagnóstico inmediato sin análisis.
- **Fila 2 — Calidad del sistema:** series temporales de `faithfulness` y `answer_relevance` de 24h con líneas de umbral de alerta; revisión semanal por el equipo de producto.
- **Fila 3 — Rendimiento:** heatmap de latencia por hora del día (detecta patrones de carga), histograma de latencia actual, throughput req/s con línea de capacidad.
- **Fila 4 — Comportamiento agéntico:** histograma de `iterations_per_task`, bar chart de `tool_call_count` por herramienta, `task_completion_rate` en ventana de 1h con umbral de alerta a 70%.
- **Fila 5 — Seguridad y costos:** contadores de rechazos por tipo de control, costo diario acumulado con proyección mensual, violations del control de autorización (valor esperado: 0).

Nota del Arquitecto: El panel de costo USD/hora en la primera fila es inusual en dashboards de operaciones de software — el costo normalmente aparece en dashboards de finanzas o de arquitectura. Lo incluyo en la primera fila de este dashboard porque en sistemas de IA el costo por petición es un SLA tan importante como la latencia: si el costo se dispara porque el agente está haciendo muchas iteraciones por una degradación del razonamiento, ese es un incidente de producción

igual que una latencia alta. El ingeniero de guardia necesita verlo en el mismo nivel de prioridad. El costo proyectado mensual en la fila de seguridad permite además anticipar si el volumen de uso actual generará una factura superior al presupuesto, antes de que sea fin de mes.

Un dashboard operativo efectivo tiene dos versiones: la versión de incidente (alta densidad, paneles correlacionados, información técnica detallada) que el ingeniero de guardia usa para diagnosticar y mitigar problemas, y la versión de revisión semanal (métricas de tendencia, comparación con períodos anteriores, indicadores de producto) que el equipo completo usa para tomar decisiones estratégicas. El dashboard descrito en esta sección es la versión de incidente; la versión de revisión semanal se construye como segundo dashboard en Grafana, compartiendo las mismas fuentes de datos pero con rangos temporales de 7 y 30 días.

Para recordar: Un dashboard operativo efectivo tiene dos versiones: una para el ingeniero de guardia durante un incidente (alta densidad de información, correlaciones visibles) y una para el equipo de producto (métricas de negocio simplificadas, tendencias de largo plazo).

Módulo 12 – Capítulo 08 – Sección 04

Alertas: configuración de umbrales y runbooks para los incidentes más probables

Una alerta sin un runbook asociado es ruido — el ingeniero que la recibe a las 3am no sabe qué hacer con ella. Un runbook sin una alerta que lo active es un documento que nadie lee hasta que ya es demasiado tarde. El par alerta + runbook es la unidad mínima de gestión de incidentes: la alerta detecta el problema, el runbook guía la resolución. El diseño de las alertas del sistema integrador parte de esta premisa: cada alerta tiene un runbook específico que el ingeniero de guardia puede seguir paso a paso sin conocimiento previo del sistema.

Las alertas del sistema se configuran en Grafana Alerting con reglas que evalúan métricas de Prometheus y Loki cada 60 segundos. El **periodo de pending** de 3 minutos previene falsos positivos por picos transitorios: una alerta solo se dispara si la condición se cumple continuamente durante 3 minutos, filtrando los spikes de latencia de 30-60 segundos causados por garbage collection de Python o por la janela de renovación de tokens JWT. Las alertas se organizan en dos severidades con canales de notificación distintos.

Las alertas **CRITICAL** despiertan al ingeniero de guardia (PagerDuty) y notifican en el canal de Slack `#incidents : error_rate > 5%` durante 3 minutos (el sistema está fallando 1 de cada 20 peticiones — impacto significativo en usuarios), `p95_latency > 5000ms` durante 3 minutos (el SLA está siendo violado con margen — el doble del SLA de 3000ms), y `dependency_unavailable`: cualquier dependencia crítica (OpenAI API, Qdrant Cloud, PostgreSQL) que falle 3 health checks consecutivos con 10 segundos de intervalo. Cada alerta CRITICAL tiene un campo `summary` con una oración que describe el problema y un campo `runbook_url` con el link al procedimiento de resolución en la documentación del Capítulo 9.

Las alertas **WARNING** notifican en Slack `#ai-ops` sin despertar a nadie: `faithfulness_ragas < 0.80` durante 15 minutos (posible drift de calidad que requiere investigación pero no acción inmediata), `p95_latency > 3500ms` durante 10 minutos (degradación de rendimiento no crítica pero que puede afectar la experiencia del usuario), y

`costo_por_hora > 1.5x_promedio_24h` durante 20 minutos (incremento anómalo de costo que puede indicar patrones de uso inusuales o ineficiencias del agente).

Las alertas **INFO** solo registran en el log del sistema sin notificaciones: `injection_detected_rate > 10/min` durante 5 minutos (posible campaña de ataque — registrar para análisis), `qdrant_collection_size > 80%_capacidad` (planificar expansión de la colección antes de que se llene), y `cert_expiry < 30_days` (los certificados TLS necesitan renovación).

La diferencia entre las alertas WARNING y CRITICAL no es solo de umbral — es de urgencia de acción. Un `faithfulness < 0.80` requiere investigación dentro de las próximas horas de trabajo, no a las 3am; un `error_rate > 5%` requiere acción inmediata porque los usuarios están experimentando fallos activos en ese momento. La calibración de qué es CRITICAL vs WARNING es una decisión de equipo que debe basarse en el impacto real en los usuarios y en la consecuencia de no actuar inmediatamente.

Configuración de alertas por severidad

- **CRITICAL (PagerDuty + Slack #incidents):** `error_rate > 5%` por 3min, `p95_latency > 5000ms` por 3min, `dependency_unavailable` (3 health checks fallidos); pending 3min para filtrar spikes transitorios.
- **WARNING (Slack #ai-ops):** `faithfulness < 0.80` por 15min, `p95_latency > 3500ms` por 10min, `costo_hora > 1.5x_promedio` por 20min; notificación sin PagerDuty — investigación dentro de horas de trabajo.
- **INFO (log only):** `injection_detected_rate > 10/min`, `collection_size > 80%`, `cert_expiry < 30d`; no interrumpe a nadie pero deja registro para revisión periódica.
- **Runbook links:** cada alerta CRITICAL y WARNING incluye `runbook_url` con el link al procedimiento específico en la documentación del Capítulo 9 — reduce el tiempo de diagnóstico inicial del ingeniero de guardia.
- **Silencing:** procedimiento de silenciado de alertas durante mantenimiento programado con duración máxima de 4 horas, solo autorizado por un lead del equipo con registro en el audit log del sistema de alertas.
- **Alertas de costo en AWS Budgets:** configurar un budget mensual en AWS Budgets con alerta al 80% del presupuesto proyectado — segunda línea de defensa para el control de costo de infraestructura.

Nota del Arquitecto: La alerta de `faithfulness < 0.80` que notifica en Slack sin despertar a nadie es una de las decisiones de diseño de alertas más importantes del sistema. Un equipo sin experiencia en IA tiende a clasificar cualquier degradación de calidad como CRITICAL porque "la calidad es importante". Pero la degradación de `faithfulness` de 0.87 a 0.79 en el sample del 5% del tráfico puede ser estadística — en el siguiente ciclo de evaluación puede volver a 0.85 sin ninguna acción. Clasificarla como CRITICAL produce alert fatigue: el equipo responde a tantas alertas CRITICAL en mitad de la noche que comienza a ignorarlas, incluyendo las que son genuinamente críticas. El criterio correcto para CRITICAL es: "si no actúo en los próximos 30 minutos, los usuarios están experimentando un impacto significativo ahora mismo". `Faithfulness` de 0.79 no pasa ese criterio; `error_rate` de 5% sí.

El sistema de alertas se prueba periódicamente mediante fire drills: el equipo simula el incidente en staging (forzando una dependency failure o inyectando latencia) y verifica que la alerta se dispara dentro del tiempo esperado, que la notificación llega al canal correcto y que el runbook permite resolver el incidente simulado. Los fire drills revelan alertas mal configuradas (thresholds incorrectos, canales de notificación equivocados) antes de que un incidente real los exponga.

Para recordar: Una alerta sin runbook es ruido — el ingeniero que la recibe a las 3am necesita saber exactamente qué comandos ejecutar para diagnosticar y mitigar el problema, sin tener que reconstruir el procedimiento desde cero bajo presión.

Módulo 12 – Capítulo 08 – Sección 05

Monitoreo de drift: detección de degradación de calidad en producción

El drift de calidad es la degradación gradual e invisible del comportamiento de un sistema de IA sin que ninguna métrica de infraestructura convencional la señale. El error rate puede mantenerse en 0.1%, la latencia P95 dentro del SLA, y los pods corriendo sin reinicios — mientras la faithfulness cae semana a semana de 0.87 a 0.83 a 0.79 porque los documentos en la base de conocimiento envejecen, los usuarios formulan preguntas que el sistema no fue diseñado para responder, o el proveedor del LLM actualiza el modelo con cambios de comportamiento no documentados. Este tipo de degradación es la más peligrosa precisamente porque no dispara las alertas convencionales — solo se detecta con monitoreo activo de métricas de calidad.

El sistema implementa monitoreo de drift en dos dimensiones: **drift de datos** (el contenido de la base de conocimiento o el perfil de queries de los usuarios cambia) y **drift de modelo** (el LLM o el modelo de embedding recibe una actualización del proveedor que cambia su comportamiento).

Para detectar **drift de datos**, el sistema ejecuta diariamente un análisis de la distribución de los reranking scores retornados por Cohere sobre una muestra de 100 queries de producción del día anterior. El reranking score medio es un proxy de la relevancia entre las queries de los usuarios y el contenido de la base de conocimiento: cuando los usuarios hacen preguntas bien cubiertas por los documentos indexados, los scores son altos (0.6-0.8); cuando los usuarios hacen preguntas sobre contenido que no está bien representado en la base de conocimiento, los scores son bajos (0.2-0.4). Una caída del score medio de más de 0.1 puntos sostenida durante 7 días es una señal de que las queries de producción están divergiendo del contenido indexado — lo que puede indicar que los usuarios están formulando preguntas sobre un área nueva que requiere nuevos documentos en la base de conocimiento.

El análisis complementario es el **clustering de embeddings de queries de producción**: cada semana, el sistema embebe una muestra de 1000 queries reales con text-embedding-3-

small y las agrupa con k-means (k=10 clusters). Los clusters que tienen un centroide muy alejado de los centroides del golden dataset de 200 pares indican categorías de preguntas reales que el golden dataset no cubre — candidatos prioritarios para expandir el dataset de evaluación. Este análisis también detecta la aparición de nuevas categorías de preguntas que no existían cuando se construyó el golden dataset.

Para detectar **drift de modelo LLM**, el sistema ejecuta el golden dataset completo de 200 pares cada 7 días y compara el faithfulness resultante contra el baseline establecido en el último deploy — almacenado en PostgreSQL en la tabla `evaluation_baselines`. Una caída de faithfulness superior a 0.05 puntos respecto al baseline genera una alerta WARNING en Slack con el mensaje "Posible drift de modelo LLM — faithfulness cayó de {baseline} a {current}". En la práctica, esto ha detectado dos actualizaciones silenciosas del modelo GPT-4o en los meses posteriores al despliegue inicial, en las que el modelo mostraba un comportamiento ligeramente diferente antes de que OpenAI publicara el changelog correspondiente.

El **knowledge staleness** es la tercera dimensión de drift: documentos en la base de conocimiento que no han sido actualizados en más de 90 días en el sistema fuente (Git, Confluence). El pipeline de ingesta compara la fecha de modificación de cada documento en la fuente con la fecha de `ingested_at` en el payload de Qdrant; los documentos con fecha de modificación en la fuente posterior a `ingested_at` en más de 7 días están desactualizados y deben re-indexarse. Los documentos con `ingested_at` superior a 90 días sin ninguna modificación en la fuente son candidatos para revisión manual — pueden ser documentos obsoletos que deben marcarse como deprecated en la base de conocimiento.

Mecanismos de detección de drift

- **Drift de datos — reranking scores:** análisis diario de la distribución de reranking scores sobre 100 queries de producción; alerta WARNING si el score_medio cae > 0.1 puntos en ventana de 7 días.
- **Drift de datos — clustering de queries:** clustering semanal de embeddings de 1000 queries reales con k-means (k=10); comparación de centroides con el golden dataset; identificación de categorías de queries no cubiertas.
- **Drift de modelo LLM:** evaluación semanal del golden dataset completo de 200 pares; alerta WARNING si faithfulness cae > 0.05 puntos vs baseline del último deploy; almacenamiento de baselines por deploy en PostgreSQL.
- **Knowledge staleness:** comparación semanal de fechas de modificación en fuentes vs `ingested_at` en Qdrant; re-indexación automática de documentos desactualizados; lista

de candidatos a deprecación para revisión manual.

- **Baseline tracking:** registro en PostgreSQL de métricas RAGAS (faithfulness, answer_relevance, context_precision, context_recall) por deploy — permite calcular deltas acumulados y detectar regresiones graduales que ningún delta puntual señalaría.

Nota del Arquitecto: El drift de modelo LLM es el más difícil de manejar porque está completamente fuera del control del equipo. Los proveedores de LLM actualizan sus modelos con mejoras de calidad y seguridad, y esas actualizaciones pueden cambiar el comportamiento del modelo en dimensiones que afectan métricas del sistema — un modelo más conservador puede tener menor faithfulness porque cita menos contexto; un modelo con mejor instruction-following puede tener mayor answer_relevance. La evaluación semanal del golden dataset es la única forma de detectar estos cambios antes de que se acumulen en una degradación significativa. Cuando se detecta drift de modelo, el equipo tiene tres opciones: ajustar el system prompt para el nuevo comportamiento del modelo, cambiar a una versión específica del modelo (si el proveedor la permite), o actualizar el baseline de evaluación si el nuevo comportamiento es mejor en la evaluación humana aunque las métricas automáticas digan lo contrario.

El monitoreo de drift convierte la observabilidad de un sistema reactivo (que detecta fallos ya ocurridos) en un sistema proactivo (que detecta degradaciones antes de que afecten a los usuarios de forma perceptible). Es la práctica que garantiza que el sistema sigue siendo útil semanas y meses después del despliegue inicial, cuando la atención del equipo se ha movido a nuevos proyectos.

Para recordar: El drift de calidad en sistemas de IA es gradual e invisible sin monitoreo activo — la calidad puede degradarse semana a semana sin que ningún indicador de error rate o latencia dispare una alerta, hasta que el usuario finalmente reporta que el sistema "ya no funciona bien".

Módulo 12 – Capítulo 08 – Sección 06

Cierre: la observabilidad como condición de mantenibilidad

El Capítulo 8 implementó la infraestructura que hace posible operar el sistema de forma sostenible en el tiempo. Sin el stack de observabilidad — las trazas en Grafana Tempo, las métricas en Prometheus, los logs estructurados en Loki, el dashboard operativo y las alertas con runbooks — el sistema puede desplegarse pero no puede mantenerse. La primera vez que el sistema tiene un incidente de producción sin observabilidad, el equipo pasa horas revisando logs sin estructura, correlacionando manualmente timestamps entre servicios distintos, y eventualmente diagnosticando la causa con un nivel de confianza que no puede justificarse. Con el stack implementado en este capítulo, el diagnóstico de la mayoría de los incidentes conocidos toma entre 5 y 15 minutos.

La dimensión de observabilidad que este sistema añade respecto a sistemas de software convencionales es la observabilidad de calidad: las métricas RAGAS sobre el sample del 5% del tráfico, el monitoreo de reranking scores para detectar drift de datos, la evaluación semanal del golden dataset para detectar drift de modelo. Estas métricas no son equivalentes a las métricas de infraestructura — no se pueden medir con Prometheus sin código custom, no se pueden calcular en milisegundos (requieren llamadas a APIs de LLM para el LLM evaluador), y no tienen umbrales absolutos que apliquen universalmente (el umbral de faithfulness depende del caso de uso). Son métricas de ingeniería de IA específicas de este dominio, y su presencia en el dashboard operativo junto con las métricas de infraestructura es lo que distingue un sistema de IA observable de uno que solo monitorea su infraestructura.

El Capítulo 9 completa el sistema con los artefactos de documentación que permiten a cualquier ingeniero del equipo operar, mantener y extender el sistema sin depender del ingeniero que lo diseñó. El runbook operativo que cada alerta del Capítulo 8 referencia se documenta en el Capítulo 9, completando el ciclo entre la detección del problema (alertas) y su resolución (runbook). Este es el vínculo entre observabilidad y operación mantenida por el equipo — la razón por la que la documentación técnica no es un capítulo aislado sino el complemento necesario del stack de observabilidad.

Lo que el Capítulo 8 implementó

- **Stack OpenTelemetry:** instrumentación automática de FastAPI/httpx/SQLAlchemy/Qdrant + spans custom del ciclo ReAct; OTEL Collector con sampling adaptativo (100% errores, 10% normal, 100% suspicious); exportación a Tempo/Prometheus/Loki.
- **Tracing distribuido:** anatomía de traza completa con spans `http.request` → `auth` → `input.validate` → `agent.run` → `tool_calls` → `llm.generate` → `output.filter`; atributo `estimated_cost_usd` custom en spans del agente.
- **Dashboard operativo:** cinco filas — estado actual (stat panels semafóricos), calidad del sistema (RAGAS series temporales), rendimiento (heatmap de latencia), comportamiento agéntico (histograma de iteraciones), seguridad y costos (rechazos + costo diario).
- **Alertas con runbooks:** CRITICAL (error_rate, latencia, dependencies) con PagerDuty + Slack; WARNING (faithfulness, latencia moderada, costo anómalo) con Slack; INFO (injection rate, collection capacity); silencing solo con autorización y registro.
- **Monitoreo de drift:** análisis diario de reranking scores, clustering semanal de queries de producción, evaluación semanal del golden dataset, knowledge staleness con re-indexación automática, baseline tracking por deploy en PostgreSQL.

Nota del Arquitecto: Una reflexión sobre el costo de implementar el stack completo de observabilidad descrito en este capítulo: el costo de los servicios de Grafana Cloud (o del stack auto-hospedado) para este sistema es de aproximadamente 150-300 USD/mes. El tiempo de implementación es de 2-3 días de ingeniería. El ROI se calcula comparando ese costo con el costo de los incidentes que el stack permite detectar y resolver más rápido. En mi experiencia, un incidente de producción de un sistema de IA sin observabilidad tarda entre 2 y 8 horas en diagnosticarse; con el stack completo, entre 10 y 45 minutos. Para un equipo de 3 ingenieros con salario de 150 USD/hora, el primer incidente que se diagnostica en 30 minutos en lugar de 4 horas ahorra 1050 USD en tiempo de trabajo — superando el costo mensual del stack de observabilidad. La observabilidad no es un costo operativo; es una inversión de ROI positivo desde el primer incidente.

Operar sin observabilidad es conducir con los ojos cerrados. La observabilidad del sistema integrador no termina en las métricas de infraestructura — requiere métricas de calidad continuas que detecten la degradación del comportamiento del modelo antes de que afecte a los usuarios.

Para recordar: La observabilidad de un sistema de IA en producción no termina en las métricas de infraestructura — requiere métricas de calidad continuas que detecten la degradación del comportamiento del modelo antes de que afecte a los usuarios.

"You cannot improve what you cannot measure." — Peter Drucker